

实验八:回溯法

班级：7 班

姓名：卢瑶

学号：20211018341

一、 实验源代码、伪代码、结果和复杂度分析

第 1 关 ： 皇后问题 在 $n \times n$ 格的棋盘上放置彼此不受攻击的 n 个皇后。按照国际象棋的规则，皇后可以攻击与之处在 同一行或同一列或同一斜线上的棋子。使用递归算法解决该问题。 伪码描述：<pre>int result[n]; bool place(int row, int column){ //当前行 和 当前列 for(int i = 0; i < row; i++){ if(column == result[i] abs(result[row] - column) == abs(row - i) return 0; //不可以放置（禁区） } return 1; } void dfs(int n, int row){ //从第 0 行开始遍历 if(n == row) //递归出口，深搜到一个解 dispSolution(n, result);</pre>

```

        for(int i = 0; i < n; i++){

            if(place(row, i)){

                result[row] == i; //记录第 i 行放置皇后的位置

                dfs(n, row + 1);

                //回溯重新初始化

                result[row] = -1;

            }

        }

    }
}

```

源代码:

```

#include <iostream>

using namespace std;
int column[999];
int cnt = 0;
//int result[999];

bool isAble(int row, int col, int n){ //判断当前位置是否可以放置皇后
    for(int i = 0; i < row; i++){ //对之前的存储情况 一一比对
        (注意理解)
        //          printf("%d %d\n", column[i], col);
        if(column[i] == col || abs(col - column[i]) ==
abs(row - i))
            return 0; //不能放
    }
    return 1; //OK
}

void print(int n){
    for(int i = 0; i < n; i++){
        for(int j = 0; j < n; j++){
            if(column[i] == j)

```

```

        printf(" Q");
    else
        printf(" *");
    }
    printf("\n");
}
printf("\n");
}

void dfs(int row, int n){ //遍历第 n 行
    if(row == n ){
        print(n);
        cnt++;
        return ;
    }

    for(int i = 0; i < n; i++){
        //记录结果
        if(isAble(row, i, n)){
            column[row] = i;
            printf("...\n");
            dfs(row + 1, n);
            //回溯重新初始化（写 x 不写都 OK）
            column[row] = 0;
        }
    }
}

int main(int argc, char *argv[]) {
    int n;
    cin >> n;
    dfs(0, n);

    cout << n << "后问题共有" << cnt << "种方案" ;
}

```

运行结果:

```

*  Q  *  *
*  *  *  Q
Q  *  *  *
*  *  Q  *

```

```

*  *  Q  *
Q  *  *  *
*  *  *  Q
*  Q  *  *

```

4皇后问题共有2种方案

时间复杂度分析：

$O(n!)$

空间复杂度分析：

$O(n)$

第2关：素数圈问题

输入正整数，把这些数字摆成一个环，要求相邻的两个数的和是一个素数。

如：长度为20的素数圈是：1 2 3 4 7 6 5 8 9 10 13 16 15 14 17 20 11 12 19 18

伪码描述：

```

int result[n];

bool place(int num,int step){//当前行 和 当前列

    if(n == step - 1 && isPrime(result[0] + num))

        return 1;

    if(n != step - 1 && isPrime(result[step - 1] + num))

```

```

        return 1;

    return 0;

}

void dfs(int n, int step){ //从第第二位开始，编号为1

    if(n == step ) //递归出口，深搜到一个解

        dispSolution(n);

    for(int i = 2; i <= n; i++){

        if(place(i, step) && !vis[i]){

            result[step] == i; //记录第 i 行放置皇后的位置

            vis[i] = 1;

            dfs(n, step + 1);

            //回溯重新初始化

            vis[i] = 0;

        }

    }

}

```

源代码：

```

#include <iostream>
#include <stdio.h>
#include <math.h>
#include <string.h>

using namespace std;

int vis[999];

```

```

int result[999];

bool isPrime(int n){
    if(n == 1)
        return 0;
    for(int i = 2; i <= sqrt(n); i++)
        if(n % i == 0) //不是素数
            return 0;
    return 1;
}

bool check(int n, int num, int step){ //判断相邻的两个数的和是否为素数
    if(step == n - 1 && isPrime(num + result[0])){
        // cout << num << endl;
        return 1;
    }
    if(step != n - 1 && isPrime(num + result[step - 1]))
        return 1;
    return 0;
}

void dfs(int n, int step){
    if(step == n ){ //递归出口(到最后一个数了，前面的数字全部都满足条件了)
        //输出结果
        for(int i = 0; i < n; i++)
            cout << result[i] << " ";
        printf("\n");
    }else{
        for(int i = 2; i <= n; i++){
            if(check(n, i, step) && !vis[i]){ //满足条件
                //且 未访问过
                result[step] = i; //记录当前步对应的数字
                vis[i] = 1; //标记为已访问过
                dfs(n, step + 1);
                //回溯 重新初始化
                vis[i] = 0;
            }
        }
    }
}

```

```
int main(int argc, char *argv[]) {
    //准备工作
    int n;
    cin >> n;
    memset(result, 0, sizeof(result));
    memset(vis, 0, sizeof(vis));
    //第一个元素一定为1
    result[0] = 1;

    dfs(n, 1); //从第二步开始遍历
    // cout << isPrime(6);
}
```

运行结果：

```
6
1 4 3 2 5 6
1 6 5 2 3 4
```

时间复杂度分析：

$O(n!)$

空间复杂度分析：

$O(n)$

二、 实验总结

1. 从 n 皇后问题中，深刻理解了回溯的思想，比如如何在解空间树上进行深度优先搜索，还有不同行、同列、同对角线时候的剪枝操作，以及最后递归出口的设定都有了进一步的提升。
2. 对于素数环问题，不难发现，和 n 皇后问题本质是一样的，无论是从代码层面还是从时间、空间复杂度层面，可以窥见，可以用回溯法解决的问题，都有相似的性质，和解题思路，以后再次遇到此类问题时，要敏锐分析，快速利用回溯法解决。